

性能优化概要-CPU

目录

- 硬件架构特性分析
 - 获取架构特性
 - Micro-benchmark
 - 与性能相关的架构组件
 - Micro-benchmark设计
- 基于Arithmetic Intensity，探索优化方向
 - 构建 Roofline Model
 - 参数
 - 在 Roofline Model 中分析计算任务瓶颈
 - Roofline Model 前置条件 & 局限性
- 基于微架构的指令级调优分享
 - 性能优化总结
 - 优化流程：
- 硬件架构信息汇总
 - 微架构信息
 - 前端
 - 后端
 - 乱序执行
 - 运算单元
 - 缓存
 - 微架构性能（指令及其对应的Latency、Throughput）
 - 计算部分
 - 访存部分

硬件架构特性分析

获取架构特性

- 厂商架构文档：存在无法获取，或信息不够齐全、准确的风险
- 基于micro-benchmark进行架构特性测试

Micro-benchmark

适用于测试单一组件或某种任务的实际性能的小型程序或代码段。

- 测试对象：尽可能从功能独立的单一架构组件入手
- 测试方法：设计基于简单代码/指令片段的小型程序
- 测试目的：充分体现架构组件的性能

与性能相关的架构组件

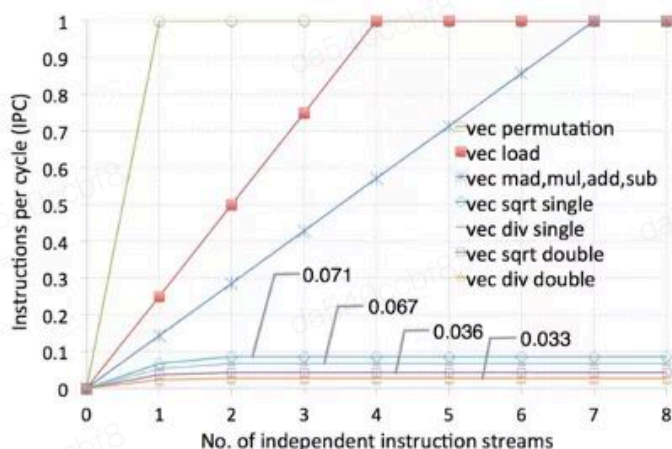
- 计算——流水线微架构（pipeline）：instruction latency & throughput
- 访存——存储层次（memory hierarchy）：global/local memory, cache latency & bandwidth
- 通信——互联（comm）：inner/intra node/cluster/core communication latency & bandwidth
- 特色架构组件

Micro-benchmark设计

- 基于独立指令流的指令吞吐（IPC）测试
- 基于依赖序列的指令延迟测试
- 基于pointer-chasing的各级访存延迟测试
- 基于stream的各级访存带宽测试
- 基于ping-pong的通信延迟测试
- 基于sync的单向延迟测试
- 基于架构问题最小复现集的自定义测试
- ...

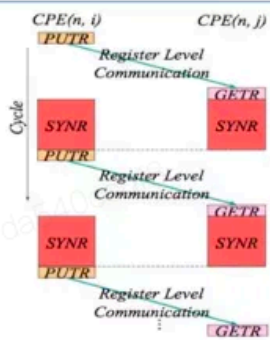
向量指令吞吐

- 以instruction per cycle (IPC) 作为度量



- At least 7 FMA to achieve peak performance.
- The throughput of SPM access and arithmetic vector instructions are all 1, except div and sqrt.
- The IPC of div and sqrt are quite low with a reciprocal throughput of 30 (1/0.033) and 28 (1/0.036) respectively in double precision, implying the succeeding independent double-precision div (sqrt) has to stall for 30 (28) cycles before it can be issued.

点对点 RLC 延迟测试



Listing 6: Kernel pseudo code of P2P RLC latency measurement

```

1 //CPE(n, i):
2 LOOP:
3 //send data to CPE j
4 putr $data, j
5 //sync in row with CPE j
6 synr $mask
7
8 //CPE(n, j):
9 LOOP:
10 //receive data from CPE i
11 getr $data
12 //sync in row with CPE i
13 synr $mask

```

Figure 7: Measuring P2P RLC

Result:

- $T_{putr}(1 \text{ cycle}) + T_{getr}(1 \text{ cycle}) + T_{data_trans} = 10 \text{ cycles} \rightarrow T_{data_trans} = 10 - 1 - 1 = 8 \text{ cycles}$
- We test every pair of CPEs in the same row/col.

RLC latency in row:

No.	IN ROW	0	1	2	3	4	5	6	7
0		/	8	8	8	9	9	9	9
1		8	/	8	8	9	9	9	9
2		8	8	/	8	9	9	9	9
3		8	8	8	/	9	9	9	9
4		8	8	8	8	/	9	9	9
5		8	8	8	8	9	/	9	9
6		8	8	8	8	9	9	/	9
7		8	8	8	8	9	9	9	/

RLC latency in col:

No.	IN COL	0	1	2	3	4	5	6	7
0		/	8	9	9	8	8	9	9
1		8	/	9	9	8	8	9	9
2		8	8	/	9	8	8	9	9
3		8	8	9	/	8	8	9	9
4		8	8	9	9	/	8	9	9
5		8	8	9	9	8	/	9	9
6		8	8	9	9	8	8	/	9
7		8	8	9	9	8	8	9	/

Table[i,j] means the latency of transferring data from i_{th} CPE to j_{th} CPE in the same ROW/COL.

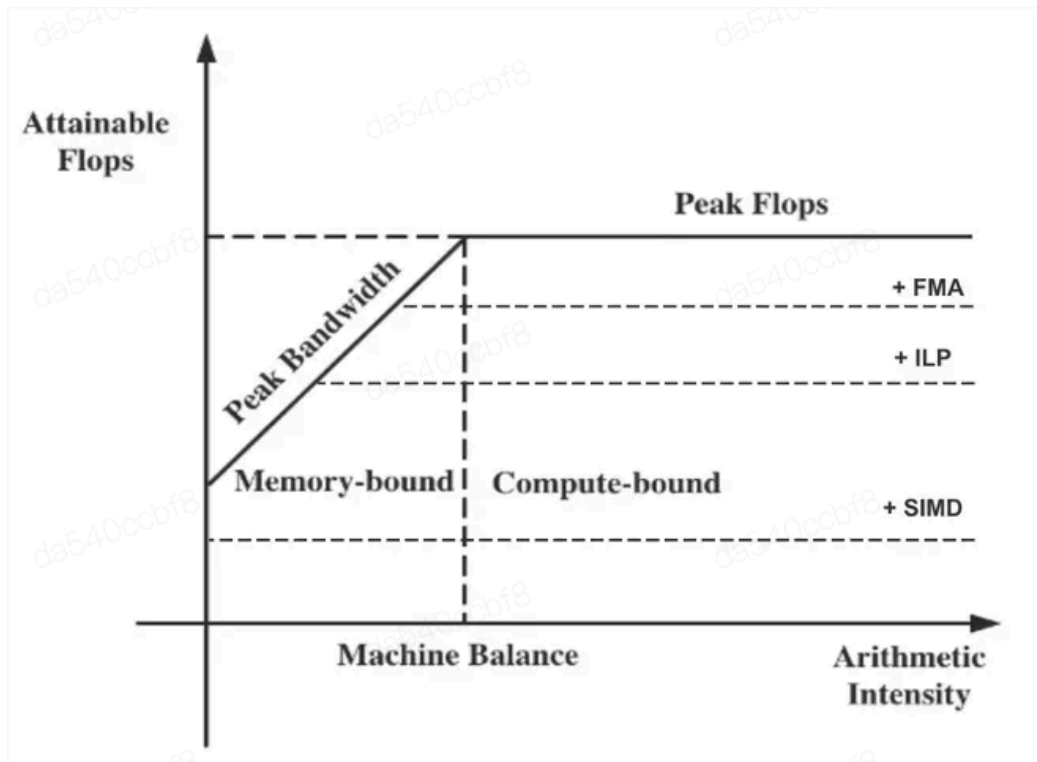
基于Arithmetic Intensity，探索优化方向

构建 Roofline Model

- 横轴(计算密度): 计算量/访存量。
- 纵轴(计算速度): 可达到的性能上限(Attainable FLOP/s, max FLOPs per Second), 取与硬件理论峰值相比的最小值。
- 斜率(内存带宽): 目标存储层次带宽(Byte/s, Max memory Access per Second)。
- 将硬件的计算/访存能力与计算任务的需求进行可视化与分析。

$$i \quad FLOP/sec = \frac{FLOP}{sec} = \frac{FLOP}{Byte} \times \frac{Byte}{sec}$$

$$FLOP/sec = AI \times BW$$



Roofline模型

参数

- 理论算力:

核数(*cores*) * 主频(*GHz*) * 128(*bits/fmla512*) * 2(*fmlaport*) * 2(*MulAndAdd*)/32(*bits_oj*)

- 内存带宽:

内存频率(*MHz*) * 64(*bit*) * 4(内存通道数)/8(*bits/Byte*)

- 计算密度:

计算密度($I(FLOP/Byte)$) = 计算量($FLOP$) ÷ 访存量($Byte$)

- 计算速度:

计算速度($FLOP/s$) = $\min$ (峰值计算速度, 计算密度 * 内存带宽)

- 计算时间:

$$\text{计算时间} = \frac{\text{计算量}}{\text{计算速度}} = \frac{\text{计算量}}{\min(\frac{\text{计算量}}{\text{访存量}} \times \text{内存带宽}, \text{理论峰值算力})}$$

$$\text{算子理论计算时间} = \begin{cases} \frac{\text{访存量}}{\text{内存带宽}} & \text{访存密集型算子} \\ \frac{\text{计算量}}{\text{理论峰值算力}} & \text{计算密集型算子} \end{cases}$$

在 Roofline Model 中分析计算任务瓶颈

- 选择正确的访存层次（global memory/LLC/local memory等）刻画Roofline Model。
- 正确计算任务的计算密度（访存层次、访存量、计算量等）。

- 将任务计算强度与Machine Balance对比，判断性能瓶颈，Memory bound/Compute bound。
- 判断性能上限，与当前实际性能进行对比。

Roofline Model 前置条件 & 局限性

- 根据是否使用FMA/SIMD/ILP选取性能上界。
- 假定计算与访存可以overlap。
- 需选取正确的存储层次带宽。
- 计算指令可以fully-pipelined (throughput-oriented model)，否则需降低性能上界 (latency bound)。
- 错误的计算性能上界/存储层次/计算访存无法掩盖将导致实际性能表现与建模存在较大偏差。
- 适合用于评估卷积/矩阵乘的并行算法性能上限与优化方向；或评估长尾算子的瓶颈类型 (memory bound/compute bound)。

i 同一计算任务在不同架构上，瓶颈类型可能不同：

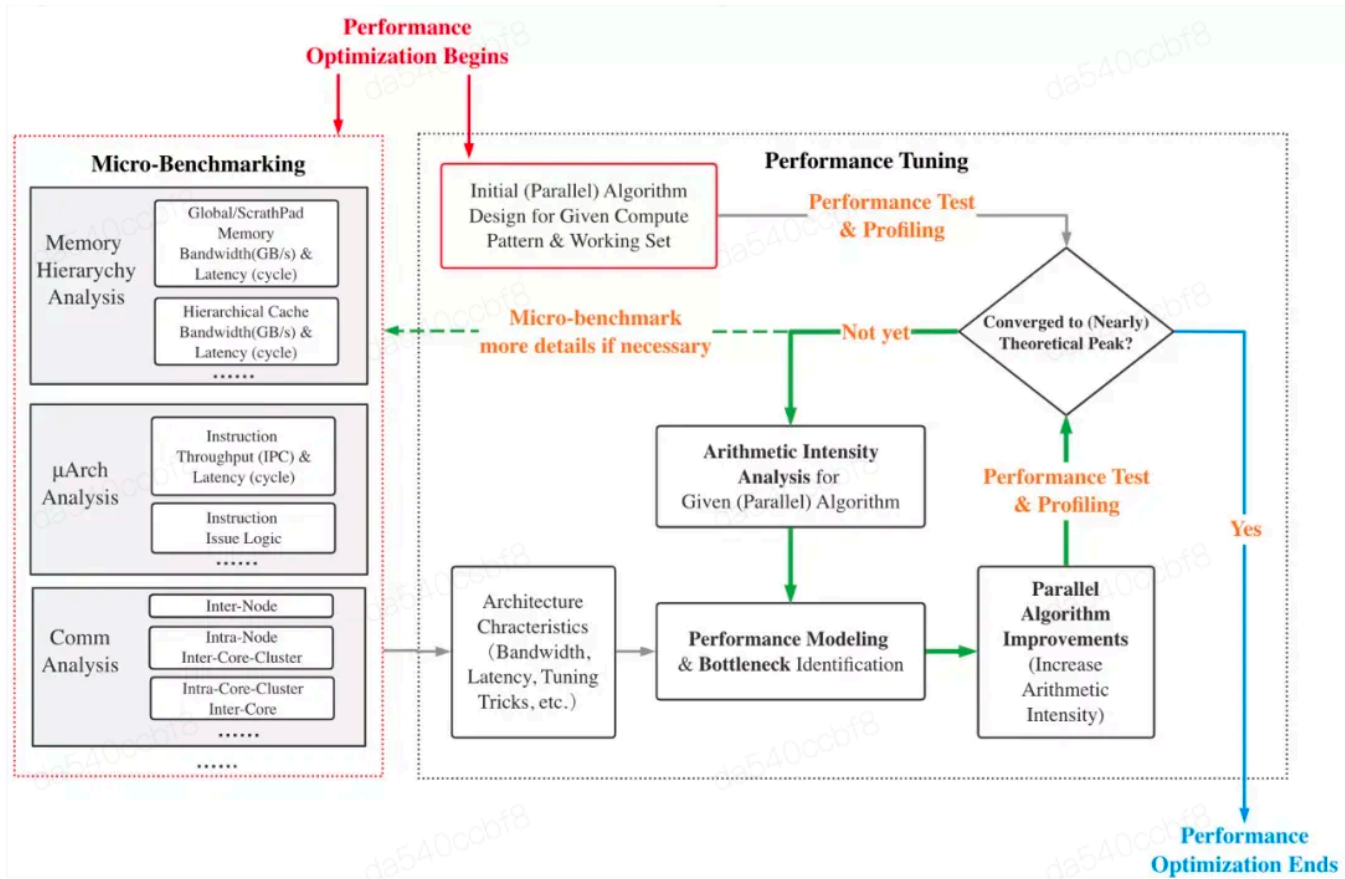
- 核心在于分析不同硬件架构的Machine Balance和任务的计算强度。
- 例如：在端侧arm平台，conv常常是compute-bound。
- 若云侧大算力处理器的local memory空间有限，或内存带宽有限，则conv将成为memory bound，优化思路截然不同。

基于微架构的指令级调优分享

Compute-bound类型的调优反映了硬件的底层架构实际的实现和代码的指令调度间的关系，调优的目标是最大化指令的吞吐，常见的方法如下：

- 最大化寄存器复用率，进而增大gemm汇编kernel的寄存器分块，隐藏ld/st指令延迟，提高指令吞吐率。
- 依据指令延迟，以及指令间依赖关系，交错指令发射时机，减少气泡。
- 寄存器双缓冲/分时复用，隐藏迭代间的数据加载开销。
- 依据指令发射端口数量和发射限制，group相关指令（尤其是VLIW类架构）。
- loop unrolling并寻找上述指令调度优化机会，提升ILP。
- 以软件模拟算法替代SFU运算（在SFU的IPC较低的情况下），并辅以loop unrolling等等。

性能优化总结



优化流程：

- **Step1** 基于micro-benchmark，分析获取架构计算/访存/通信特性
- **Step2** 结合计算任务的计算模式，设计并行算法
- **Step3** 分析性能瓶颈，结合架构特性，评估性能是否达到预期，或返回Step2改善并行算法设计，针对性调优计算/访存瓶颈，迭代直至性能收敛

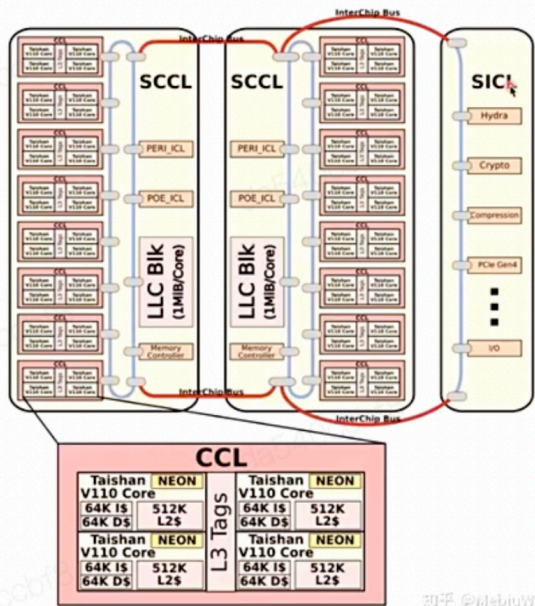
硬件架构信息汇总

Arm Server 微架构特点——以鲲鹏920处理器为例



鲲鹏920处理器的体系结构特点——片上系统

- 处理器内核集群 CCL (Core Cluster)
 - 包含4个处理器内核及其L1 I/DCache、L2 Cache
 - 包含1个L3 Cache Tag
- 超级内核集群 SCCL (Super CCL)
 - 包含6个/8个内核集群 (CCL)
 - 包含2个I/O集群，4个DDR控制器
 - 包含若干L3 Cache Data，多核缓存一致性模块(HHA)
 - 环形总线互联
- 相应I/O部件：I/O集群，超级I/O集群



微架构信息

前端

- 每周期取值x条
- 是否支持静态、动态分支预测

后端

乱序执行

运算单元

i 关注流水线的宽度

- 整数运算单元：x条ALU流水线，x条MDU流水线（整数乘法除法运算）。
- Adv SIMD与浮点运算单元：x条FSU流水线。
- Load/Store单元：x个L/S端口，是否包含硬件自动预取器。

缓存

</> 缓存信息

Plain Text | 收起 ^

- L1 ICache：x路，xx KiB (private)
- L1 DCache：x路，xx KiB (private)
- L2 Cache： x路，xxx KiB (private)
- L1 ITLB： 全关联，xx表项（支持xKB~xGB页大小）

- 5 • L1 DTLB: 全关联, xx表项 (支持xKB~xGB页大小)
- 6 • L2 TLB: x路组关联, xxxx表项
- 7 • LLC/L3 Cache: xx~xxxMiB (x MiB per core, shared)

微架构性能（指令及其对应的Latency、Throughput）

计算部分

 硬件手册中查阅对应信息。

Instruction	Latency (cycles)	Throughput (IPC)
IADD (vector)	2	2
FADD (vector)	5	2
FMUL (vector)	5	2
FMADD (scalar)	5	2
FMLA (vector)	5	2

如上，需要 $5 \times 2 = 10$ 条（无数据依赖）的FMLA指令，才能充分利用Neon计算单元（Adv SIMD运算单元）

访存部分

 与前者类似

Instruction	Latency (cycles)	Throughput (IPC)
LDR/LD1 (16 bytes)	~4 (L1 cache hit)	<u>1.85</u>
STR/ST1 (16 bytes)		<u>0.95</u>